

speckit.analyze

User Input

\$ARGUMENTS

You **MUST** consider the user input before proceeding (if not empty).

Pre-Execution Checks

Check for extension hooks (before analysis): - Check if .specify/extensions.yml exists in the project root. - If it exists, read it and look for entries under the hooks.before_analyze key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where enabled is explicitly false. Treat hooks without an enabled field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook condition expressions: - If the hook has no condition field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty condition, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its optional flag: - **Optional hook** (optional: true): ``` ## Extension Hooks

****Optional Pre-Hook**:** {extension}

Command: `/{command}`

Description: {description}

Prompt: {prompt}

To execute: `/{command}`

```

- **Mandatory hook** (optional: false):

## Extension Hooks

**\*\*Automatic Pre-Hook\*\*:** {extension}

Executing: `/{command}`

EXECUTE\_COMMAND: {command}

Wait for the result of the hook command before proceeding to the Goal.

After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal {command} id shown above, e.g. a skills-mode agent runs it as `/skill:speckit-...` or `$speckit-...`). Emitting the block alone does not run the hook.

- If no hooks are registered or `.specify/extensions.yml` does not exist, skip silently

## Goal

Identify inconsistencies, duplications, ambiguities, and underspecified items across the three core artifacts (`spec.md`, `plan.md`, `tasks.md`) before implementation. This command **MUST** run only after `/speckit.tasks` has successfully produced a complete `tasks.md`.

## Operating Constraints

**STRICTLY READ-ONLY:** Do **not** modify any files. Output a structured analysis report. Offer an optional remediation plan (user must explicitly approve before any follow-up editing commands would be invoked manually).

**Constitution Authority:** The project constitution (`.specify/memory/constitution.md`) is **non-negotiable** within this analysis scope. Constitution conflicts are automatically **CRITICAL** and require adjustment of the spec, plan, or tasks—not dilution, reinterpretation, or silent ignoring of the principle. If a principle itself needs to change, that must occur in a separate, explicit constitution update outside `/speckit.analyze`.

## Execution Steps

### 1. Initialize Analysis Context

Run `.specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks` once from repo root and parse JSON for `FEATURE_DIR` and `AVAILABLE_DOCS`. Derive absolute paths:

- `SPEC = FEATURE_DIR/spec.md`
- `PLAN = FEATURE_DIR/plan.md`

- TASKS = FEATURE\_DIR/tasks.md

Abort with an error message if any required file is missing (instruct the user to run missing prerequisite command). For single quotes in args like “I’m Groot”, use escape syntax: e.g. ‘I’’m Groot’ (or double-quote if possible: “I’m Groot”).

## 2. Load Artifacts (Progressive Disclosure)

Load only the minimal necessary context from each artifact:

### From spec.md:

- Overview/Context
- Functional Requirements
- Success Criteria (measurable outcomes — e.g., performance, security, availability, user success, business impact)
- User Stories
- Edge Cases (if present)

### From plan.md:

- Architecture/stack choices
- Data Model references
- Phases
- Technical constraints

### From tasks.md:

- Task IDs
- Descriptions
- Phase grouping
- Parallel markers [P]
- Referenced file paths

### From constitution:

- Load .specify/memory/constitution.md for principle validation

## 3. Build Semantic Models

Create internal representations (do not include raw artifacts in output):

- **Requirements inventory:** For each Functional Requirement (FR-###) and Success Criterion (SC-###), record a stable key. Use the explicit FR-/SC- identifier as the primary key when present, and optionally also derive an imperative-phrase slug for readability (e.g., “User can upload file” → user-can-upload-file). Include only Success Criteria items that require buildable work (e.g., load-testing infrastructure, security audit

tooling), and exclude post-launch outcome metrics and business KPIs (e.g., “Reduce support tickets by 50%”).

- **User story/action inventory:** Discrete user actions with acceptance criteria
- **Task coverage mapping:** Map each task to one or more requirements or stories (inference by keyword / explicit reference patterns like IDs or key phrases)
- **Constitution rule set:** Extract principle names and MUST/SHOULD normative statements

## **4. Detection Passes (Token-Efficient Analysis)**

Focus on high-signal findings. Limit to 50 findings total; aggregate remainder in overflow summary.

### **A. Duplication Detection**

- Identify near-duplicate requirements
- Mark lower-quality phrasing for consolidation

### **B. Ambiguity Detection**

- Flag vague adjectives (fast, scalable, secure, intuitive, robust) lacking measurable criteria
- Flag unresolved placeholders (TODO, TKTK, ???, <placeholder>, etc.)

### **C. Underspecification**

- Requirements with verbs but missing object or measurable outcome
- User stories missing acceptance criteria alignment
- Tasks referencing files or components not defined in spec/plan

### **D. Constitution Alignment**

- Any requirement or plan element conflicting with a MUST principle
- Missing mandated sections or quality gates from constitution

### **E. Coverage Gaps**

- Requirements with zero associated tasks
- Tasks with no mapped requirement/story
- Success Criteria requiring buildable work (performance, security, availability) not reflected in tasks

## F. Inconsistency

- Terminology drift (same concept named differently across files)
- Data entities referenced in plan but absent in spec (or vice versa)
- Task ordering contradictions (e.g., integration tasks before foundational setup tasks without dependency note)
- Conflicting requirements (e.g., one requires Next.js while other specifies Vue)

## 5. Severity Assignment

Use this heuristic to prioritize findings:

- **CRITICAL:** Violates constitution MUST, missing core spec artifact, or requirement with zero coverage that blocks baseline functionality
- **HIGH:** Duplicate or conflicting requirement, ambiguous security/performance attribute, untestable acceptance criterion
- **MEDIUM:** Terminology drift, missing non-functional task coverage, underspecified edge case
- **LOW:** Style/wording improvements, minor redundancy not affecting execution order

## 6. Produce Compact Analysis Report

Output a Markdown report (no file writes) with the following structure:

### Specification Analysis Report

| ID | Category    | Severity | Location(s)      | Summary                      | Recommendations                      |
|----|-------------|----------|------------------|------------------------------|--------------------------------------|
| A1 | Duplication | HIGH     | spec.md:L120-134 | Two similar requirements ... | Merge phrasing; keep clearer version |

(Add one row per finding; generate stable IDs prefixed by category initial.)

### Coverage Summary Table:

| Requirement Key | Has Task? | Task IDs | Notes |
|-----------------|-----------|----------|-------|
|-----------------|-----------|----------|-------|

**Constitution Alignment Issues:** (if any)

**Unmapped Tasks:** (if any)

## Metrics:

- Total Requirements
- Total Tasks
- Coverage % (requirements with  $\geq 1$  task)
- Ambiguity Count
- Duplication Count
- Critical Issues Count

## 7. Provide Next Actions

At end of report, output a concise Next Actions block:

- If CRITICAL issues exist: Recommend resolving before / `speckit.implement`
- If only LOW/MEDIUM: User may proceed, but provide improvement suggestions
- Provide explicit command suggestions: e.g., "Run / `speckit.specify` with refinement", "Run /`speckit.plan` to adjust architecture", "Manually edit `tasks.md` to add coverage for 'performance-metrics'"

## 8. Offer Remediation

Ask the user: "Would you like me to suggest concrete remediation edits for the top N issues?" (Do NOT apply them automatically.)

## 9. Check for extension hooks

After reporting, check if `.specify/extensions.yml` exists in the project root. - If it exists, read it and look for entries under the `hooks.after_analyze` key - If the YAML cannot be parsed or is invalid, skip hook checking silently and continue normally - Filter out hooks where `enabled` is explicitly `false`. Treat hooks without an `enabled` field as enabled by default. - For each remaining hook, do **not** attempt to interpret or evaluate hook condition expressions: - If the hook has no condition field, or it is null/empty, treat the hook as executable - If the hook defines a non-empty condition, skip the hook and leave condition evaluation to the HookExecutor implementation - For each executable hook, output the following based on its optional flag: - **Optional hook** (`optional: true`): ``  
## Extension Hooks

```
Optional Hook: {extension}
Command: `{command}`
Description: {description}
```

```
Prompt: {prompt}
```

To execute: `{command}`  
`{command}`

- **Mandatory hook** (optional: false):

```
Extension Hooks
```

```
Automatic Hook: {extension}
```

```
Executing: `{command}`
```

```
EXECUTE_COMMAND: {command}
```

After emitting the block above you **MUST** actually invoke the hook and wait for it to finish before continuing. Run it the same way you would run the command yourself in this agent/session (the invocation may differ from the literal {command} id shown above, e.g. a skills-mode agent runs it as / skill:speckit-... or \$speckit-...). Emitting the block alone does not run the hook.

- If no hooks are registered or .specify/extensions.yml does not exist, skip silently

## Operating Principles

### Context Efficiency

- **Minimal high-signal tokens:** Focus on actionable findings, not exhaustive documentation
- **Progressive disclosure:** Load artifacts incrementally; don't dump all content into analysis
- **Token-efficient output:** Limit findings table to 50 rows; summarize overflow
- **Deterministic results:** Rerunning without changes should produce consistent IDs and counts

### Analysis Guidelines

- **NEVER modify files** (this is read-only analysis)
- **NEVER hallucinate missing sections** (if absent, report them accurately)
- **Prioritize constitution violations** (these are always CRITICAL)
- **Use examples over exhaustive rules** (cite specific instances, not generic patterns)
- **Report zero issues gracefully** (emit success report with coverage statistics)

# Context

\$ARGUMENTS